RAG Assisted Chef Agent

Arin Agarwal*
aa5844@columbia.edu
Columbia University in the City of New York
Seattle, Washington, USA

Abstract

We present a culinary generation agent that combines chemical flavor knowledge, retrieval-augmented generation (RAG), and large language models (LLMs) to design and evaluate novel recipes. Traditional recipe generation systems rely primarily on language models trained on existing recipes, which limits their ability to reason about underlying flavor compatibility. Our approach integrates a structured flavor graph derived from odorant–ingredient relationships with textual culinary knowledge sources, including cookbooks and flavor reference texts. The agent first generates candidate recipes and identifies prominent ingredients. The recipes are generated in accordance to parsed user intent, and must only contain ingredients that the user has in their pantry (pantry info is provided to the LM). It then retrieves relevant context from a vector database containing odorant descriptions, ingredient flavor profiles, and cookbook passages. Using this context, the system proposes ingredient additions grounded in both chemical aroma overlap and culinary practice. The agent subsequently synthesizes improved recipes incorporating these additions and evaluates them using a multi-signal scoring framework combining odorant compatibility, retrieval evidence from culinary texts, flavor diversity heuristics, and language model judgment. Experiments demonstrate that integrating chemical flavor knowledge with retrieval-based culinary context produces recipes with stronger predicted flavor coherence compared to generation based solely on language models. This work illustrates how combining structured food chemistry data with LLM-based reasoning can support more principled computational creativity in culinary design.

Keywords: Large Language Model, Retrieval Augmented Generation, Bipartite Flavor graph, Odorant

Link to demonstration video
Link to github

1 Problem Description

Creating new recipes combines culinary intuition, cultural knowledge, and an implicit understanding of flavor chemistry. Skilled chefs often reason about ingredients in terms of shared aromas, complementary textures, and balanced flavor profiles. Many recent computational approaches to this problem rely on large language models (LLMs) trained on large corpora of existing recipes, which can produce plausible dishes but lack principled mechanisms for evaluating flavor compatibility or systematically exploring novel ingredient combinations.

Research in chemistry and gastronomy as shown that up to 80% of what we taste comes from what we smell. The volatile compounds that produce the smells of different ingredients are known as odorants. There are generally three ways that odorants can be combined together to enhance the flavor of a dish.

- Ingredients with similar odorants complement each other, and enhance each others flavor
- Two different odorants can pair well together if they each make up for things that they lack. Think about this like adding pickles to a grilled cheese sandwich. The acidity and freshness of the pickles contrasts with the oiliness of the cheese and creates a more balanced and harmonious dish.
- Two opposite ingredients can be brought together with a "bridge" ingredient that shares odorants with the other two.

In this work, we introduce a culinary agent that integrates flavor chemistry data with language model reasoning to generate and evaluate novel recipes. We designed this agent to be useful and accessible for chefs of any level. It can take queries with varying amounts of culinary technical difficulty and apply its knowledge to generate multiple valid recipes. It can then evaluate them against each other to find which recipe would likely taste the best.

2 System architecture

The culinary agent is designed as a multi-stage pipeline that integrates large language models (LLMs), structured flavor knowledge, and retrieval-based reasoning to generate and evaluate novel recipes. The system transforms a user request and a set of available ingredients into optimized recipes through a sequence of generation, reasoning, and evaluation stages. You can see the overall architecture in the diagram below.

2.1 Input and Constraint Processing

The system begins by processing user input describing a desired dish or set of preferences. This input is parsed into two categories of constraints. **Hard constraints** represent requirements that must be satisfied, such as dietary restrictions, nutritional limits, or specific ingredient availability. **Soft constraints** represent stylistic or flavor preferences such as spiciness, sweetness, or cuisine type. These constraints

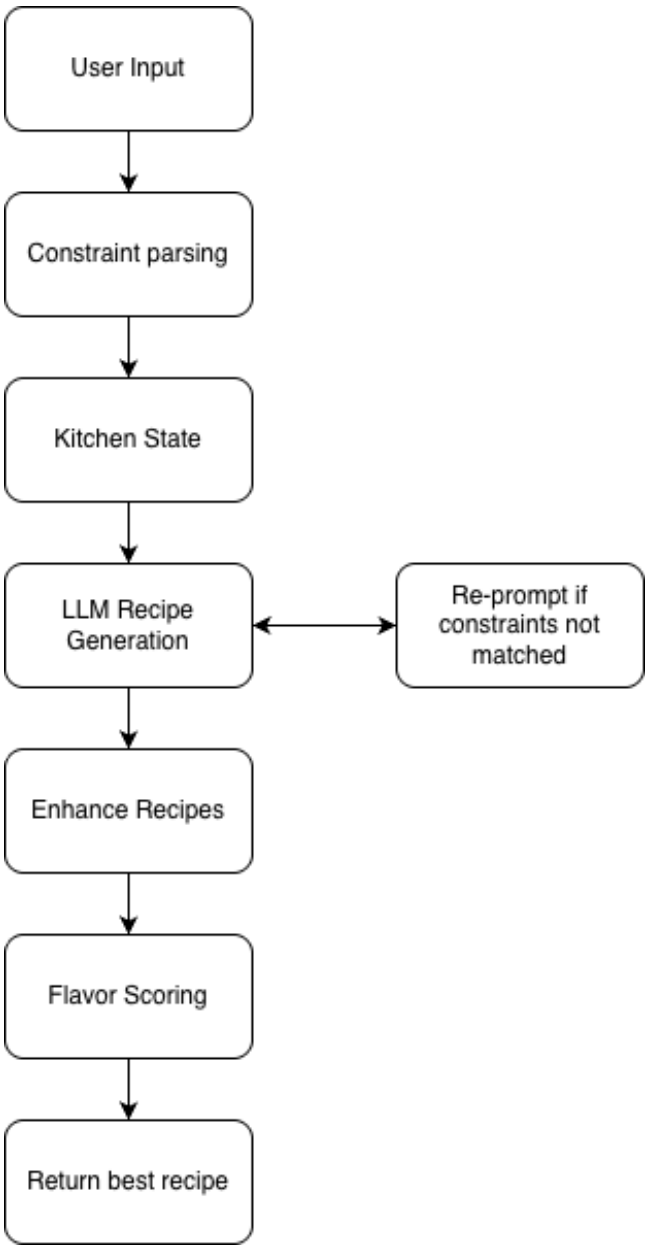

Figure 1. Overall Architecture

are combined with the **kitchen state,** a representation of the ingredients, cookware, and pantry items available to the user.

2.2 Candidate Recipe Generation

Given the kitchen state and parsed constraints, the system uses an LLM to generate a set of 3 candidate recipes. The model is prompted to identify the most prominent ingredients and flavor components in the dish and to produce structured recipe candidates that satisfy the input constraints. This stage produces multiple diverse candidate recipes, each containing a core ingredient set and an initial flavor profile.

The generated recipes are then checked against the user intent again to ensure that they meet the constraints. There is a re-prompting loop that checks each recipe, and then prompts the LLM to regenerate it if there are any problems.

2.3 Flavor Reasoning and Ingredient Expansion

To enhance the generated recipes, the system performs flavor reasoning using both structured flavor data and culinary knowledge retrieved from text corpora. A **flavor graph** constructed from odorant–ingredient relationships identifies ingredients that share key aromatic compounds with those in the candidate recipe. This graph-based reasoning suggests ingredients that are chemically compatible with the dish.

In parallel, a **retrieval-augmented generation (RAG)** module queries a vector database built from culinary sources, including cookbooks and flavor reference texts. Retrieved passages provide contextual information about traditional pairings, preparation methods, and complementary ingredients. The LLM integrates these sources to propose ingredient additions that enhance flavor balance and complexity.

2.4 Recipe Construction and Scoring

For each candidate recipe, the system generates an expanded recipe incorporating recommended ingredient additions. The resulting recipes are evaluated using a multi-factor scoring framework:

- **odorant score** is computed using a graph that connects ingredients to odorants that they contain. A higher score means that the ingredients in the recipe share more odorants
- **rag score** is computed for every pair of ingredients in the recipe. It uses vector search over around 70 cookbooks from different cuisines and gives a combination of ingredients points for being mentioned together in real life recipes.
- **llm score** is computed by asking an llm to score a dish on a scale of 1-100 based on flavor balance and culinary realism.

Each score is normalized to be between 0-1, then the following formula is used to compute overall score out of 100.

$$S_{\text{overall}} = 30 * S_{\text{odorant}} + 30 * S_{\text{rag}} + 40 * S_{\text{llm}}$$

3 Agent Design

3.1 Intent Parsing

To mitigate LLM hallucination and ensure that the user gets a recipe that fits their needs, the agent spends a lot of time validating and parsing the user intent. In experiments, we often saw that the LLM was generating recipes that didn't fit the prompt completely. Parsing user intent into defined

categories allows the agent to check what the LLM generates against concrete data.

3.2 Hybrid Generation

A central design choice in the system is the integration of structured flavor data with language model reasoning. Purely generative approaches can produce plausible recipes but often lack consistency in flavor pairings or ingredient compatibility. To address this limitation, the agent incorporates a **flavor graph derived from odorant–ingredient relationships**, enabling the system to reason about flavor compatibility at a chemical level. Ingredients sharing key odorant compounds are more likely to produce harmonious flavor combinations, and this information guides the expansion of candidate recipes.

3.3 Language Model

After experimentation with a few models, we chose to use llama-3.3-70b-versatile model. We initially started with a hugging face model that could be loaded and run directly on the colab GPU. Generation with a local model took too long, so we moved to using a llama model through groq. We found that the llama 70B parameter model gave us a perfect balance of speed and useful results. Smaller models tended to hallucinate more, and would not output things in the format that we wanted.

4 Data and RAG

We separated our data into a corpus geared around ingredient pairings, and a different corpus centered around recipe generation. All of our data was preprocessed by converting it into text files, segmenting it into overlapping chunks, embedding it using a bge sentence model, and finally indexing it using FAISS. We chose to use FAISS so that we can have **fast retrieval of data** during runtime

4.1 Ingredient Pairing

The culinary agent relies on both structured chemical flavor data and unstructured culinary text to support ingredient reasoning and recipe generation. To provide this knowledge to the language model in a usable form, we construct two complementary resources: (1) a structured flavor knowledge graph derived from odorant–ingredient relationships and (2) a retrieval corpus used in a retrieval-augmented generation (RAG) pipeline. Together, these datasets allow the system to reason about ingredient compatibility both chemically and contextually.

A primary structured data source for the system is **FlavorDB**, a publicly available dataset that catalogs the chemical compounds responsible for the aroma of foods. Each food item in FlavorDB is associated with a set of odorant molecules that contribute to its perceived flavor profile. Because many foods share common odorants, this information can be used to infer potential ingredient pairings based on chemical similarity.

To support efficient reasoning over the FlavorDB dataset, the raw data was parsed into structured JSON representations. Two complementary mappings were constructed:

- **Food-to-odorant mapping**, which associates each ingredient with the set of odorant compounds present in that food.
- **Odorant-to-food mapping**, which lists all ingredients containing a given odorant compound.

These mappings allow the system to quickly identify ingredients that share key aroma molecules with those present in a candidate dish. During recipe refinement, the agent queries these structures to identify potential ingredient additions that maximize odorant overlap with the existing ingredient set.

This structured representation effectively forms a **bipartite flavor graph** connecting ingredients and odorant compounds. Traversing this graph allows the system to propose flavor pairings based on chemical similarity rather than purely textual association.

In addition to the structured odorant graph, the system also generates RAG-friendly explanatory documents derived from the FlavorDB data. These documents provide natural-language descriptions of ingredient flavor profiles and the odorant compounds that contribute to them. For example, a document describing a specific ingredient may list key aroma molecules and explain their sensory characteristics (e.g., citrus-like, floral, smoky).

Creating these explanatory documents serves two purposes. First, it allows the language model to access chemical flavor information through the same retrieval interface used for cookbook knowledge. Second, it enables the model to generate interpretable explanations when recommending ingredient additions, linking proposed pairings to shared aromatic properties.

In addition to this data about odorants, we wanted to add less academic data into our flavor pairing corpus. For this we used the book *The Flavor Bible*. This book is highly influential in the culinary world, and contains information about thousands of ingredient parings that are used in practice.

4.2 Recipe Generation

Once the agent identifies ingredients to enhance a dish, we wanted to ensure that the ingredients were added into the dish in the best possible way. To do this, we created a second RAG corpus with around 70 cookbooks from different cuisines. This gives us a database of over 10,000 recipes that can be referenced as the LM is creating new recipes.

5 Experiments and Examples

Using a basic query and a more advanced query, we can demonstrate the agent can appeal to both novice and experienced chefs.

5.1 Basic Query

Based on a kitchen state that includes things in my own pantry, I prompted my agent to create a high protein dinner for me. The initial recipe that was given to me by my llama model was grilled salmon with spinach. After running the agent was completed, the new recipe was Grilled Salmon with Spinach and Citrus Herb Butter. There is also a list of other ingredients not in my pantry that would greatly enhance the dish, such as mango, avocado, pineapple, and ginger.

5.2 Advanced Query

We used the following query for the advanced section: *Give me recipes that include an emulsified sauce with tarragon.* We see here that all 3 recipes generated include tarragon, and are emulsification based sauces. We run similar tests to ascertain that the agent can produce strictly vegetarian, gluten free, low-carb, or recipes that include or exclude an ingredient.

5.3 Recommendation Accuracy

We conducted an experiment where we removed 2 of the three ingredients that were added to a recipe, and then re-prompted the Agent to find ingredient pairings that would go with the recipe. We found that the agent rediscovered the removed ingredients close to 90% of the time.

5.4 Future Experiments

We will conduct experiments where the odorant compatibility of LLM only recipe generation is compared with odorant compatibility of the agent generated recipes. We hope to see that there are a significantly higher proportion of compatible odorants in the agent generated recipes.

We also hope to perform an ablation study where we remove different parts of the agent and see how the outcome changes. This will allow us to prove the different architectural decisions that we made.

6 Limitations

Besides the occasional json parsing error, there are a few limitations that we would like to address with the current agent.

6.1 Contrasting and Bridge Odorants

Currently, the agent only looks at odorants shared across ingredients to determine ingredient pairings. However, as mentioned in the problem statement, this is not the only way to pair ingredients. It is possible for contrasting odorants to create harmonious flavors. Using resources like cookbooks and *The Flavor Bible* is currently the only way that our agent recommends ingredients that don't share a high percentage of odorants. In the future, we want to develop a way to use chemistry and gastronomy to recommend contrasting odorants for recipes.

6.2 Odorant Concentrations

We were unable to find a dataset telling us about the absolute concentrations of odorants in different ingredients. Currently, we roughly know what the most impactful odorants in an ingredient are, but we lack actual composition data.

7 Future Work and Use

There are several directions for improving this culinary agent in future iterations. First, the flavor reasoning component could be expanded with richer chemical and sensory datasets beyond the current odorant sources. As discussed in the limitations section, we should be able to find contrasting odorants that enhance dishes. Gathering this data and implementing it will be a major point of future work.

Another promising direction is improving the generation pipeline itself. Instead of producing recipes in a single pass, the agent could use a multi-step refinement process where candidate recipes are generated, evaluated for flavor compatibility using the odorant graph, and iteratively improved.

We see this type of agent being used in future commercial kitchens as well. This would require cameras in pantries to record available ingredients and set kitchen state. Chefs would be able to interact with the agent and prompt it to help them devise new, better recipes. A future extension to the agent could be an Augmented Reality enabled repromting loop, that allows the agent to help chefs out when they are cooking and make a mistake. For example, the agent could monitor how much salt the chef is adding. If they add too much, the agent could offer corrections that keep the recipe taste intact.